

EAS 550 – Transformation, Quality, and CI/CD

Perishable Inventory Intelligence Platform: Phase 2

Sidharth Kirit Saholiya
Akaash Ramanathan Vontivillu
Harish Harikrishnabhai Sondagar
Riddhi Jaysukhbhai Vaghani

Abstract—Phase 1 built out a normalized OLTP schema for the Perishable Inventory Intelligence Platform. In Phase 2, we took that schema and turned it into something actually useful for analytics. That meant building a Star Schema with dbt, writing tests to catch bad data, putting together some more complex SQL queries with performance improvements, and wiring up a CI/CD pipeline through GitHub Actions.

I. OVERVIEW

Phase 1 gave us a 3NF-normalized relational schema on Neon PostgreSQL, covering regions, stores, categories, products, suppliers, promotions, and inventory transactions. Phase 2 had three main things we needed to get done: build the dimensional model on top of what was already there, set up data quality checks with automated tests, and write more complex analytical queries with some actual performance tuning done on them.

II. ENVIRONMENT SETUP

We set dbt up in a Python 3.11 virtual environment, mostly because of some compatibility issues we ran into with certain packages. After installing dbt-postgres, we ran dbt init to create the project and configured the Neon PostgreSQL connection in profiles.yml. Running dbt debug confirmed the connection worked. For the CI/CD side, we injected database credentials through GitHub Secrets so nothing sensitive ever gets committed to the repo.

III. STAR SCHEMA DESIGN

A. Motivation

The 3NF schema from Phase 1 is fine for writes, but running analytics on it is slow because you end up joining a ton of tables together every time. A Star Schema fixes this by putting everything around a central fact table with flat dimension tables attached to it. Fewer joins, faster aggregations.

B. Schema Structure

We ended up with one fact table and three dimension tables:

- **fact_inventory** – transaction-level data: transaction_id, product_id, store_id, supplier_id, transaction_date, expiration_date, quantity, unit_price, waste_amount, profit, is_promotion, demand_level.

- **dim_product** – product and category info combined: product_code, product_name, category_name, shelf_life_days, storage_temp_celsius, spoilage_sensitivity.
- **dim_store** – store and region: store_code, region_name.
- **dim_supplier** – supplier info: supplier_code, supplier_score.

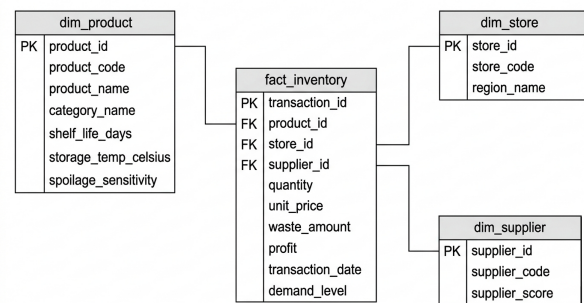


Fig. 1. Star Schema Design for the Data Warehouse

IV. DBT MODELS

The dbt models are stored in dbt_project/models/example/. This folder contains dim_product.sql, dim_store.sql, dim_supplier.sql, fact_inventory.sql, and schema.yml.

The models read directly from the Phase 1 OLTP tables using standard SQL table references.

dim_product.sql joins products and categories to flatten product and storage attributes into one table.

dim_store.sql joins stores and regions so region_name is right there without needing another join later.

dim_supplier.sql pulls supplier attributes directly from the suppliers source.

fact_inventory.sql grabs transaction-grain fields from inventory_transactions, including dates, quantities, pricing, the promotion flag, waste, and profit.

All models were materialized by running:

```
dbt run
```

V. DATA QUALITY TESTING

Tests are defined in `dbt_project/models/example/schema.yml` using dbt's built-in test types:

- **not_null** – on primary keys, foreign keys, and any columns that analytics actually depends on.
- **unique** – on dimension primary keys, business codes, and transaction IDs.
- **relationships** – on `product_id`, `store_id`, and `supplier_id` in `fact_inventory` to make sure referential integrity holds against the dimension tables.

For example, `product_id` in `fact_inventory` is validated against `dim_product`, and `store_id` and `supplier_id` are checked against `dim_store` and `dim_supplier` respectively. Tests are run with:

```
dbt test
```

All tests passed, including null checks, uniqueness checks, and referential integrity checks across the dimensional model.

A. Data Catalog

We generated docs with `dbt docs generate`. You can serve them locally using `dbt docs serve`. The catalog has model descriptions, column metadata, and test information for the warehouse models.

VI. ADVANCED ANALYTICAL QUERIES

Three analytical queries are in `dbt_project/analyses/`, each using more complex SQL constructs.

A. Waste Analysis

This query uses a single CTE to aggregate `waste_amount` by product and then joins the result with `dim_product` to show product names.

```
WITH waste_cte AS (  
  SELECT  
    product_id,  
    SUM(waste_amount) AS total_waste  
  FROM fact_inventory  
  GROUP BY product_id  
)  
  
SELECT  
  p.product_name,  
  w.total_waste  
FROM waste_cte AS w  
INNER JOIN dim_product AS p  
  ON w.product_id = p.product_id  
ORDER BY w.total_waste DESC;
```

B. Supplier Profit Ranking

This query ranks suppliers by total profit using `RANK()` `OVER()` on aggregated `fact_inventory` data.

```
SELECT  
  supplier_id,  
  SUM(profit) AS total_profit,  
  RANK() OVER (ORDER BY SUM(profit) DESC) AS  
    supplier_rank  
FROM fact_inventory  
GROUP BY supplier_id;
```

C. Demand Trend Moving Average

This query calculates a moving average of quantity per product over time using a window function on `fact_inventory`.

```
SELECT  
  product_id,  
  transaction_date,  
  quantity,  
  AVG(quantity) OVER (  
    PARTITION BY product_id  
    ORDER BY transaction_date  
    ROWS BETWEEN 7 PRECEDING AND CURRENT ROW  
  ) AS moving_avg_quantity  
FROM fact_inventory;
```

VII. PERFORMANCE TUNING

A. Baseline Measurement

We ran `EXPLAIN ANALYZE` on the moving-average query first. Without any indexing, the planner did a Sequential Scan over `inventory_transactions`, going through every row even though the query only cares about specific products and dates. Execution time was around 851 ms.

B. Index Creation

We added a composite index on the two columns the window function depends on:

```
CREATE INDEX idx_fact_product_date  
ON inventory_transactions (product_id,  
  transaction_date);
```

`product_id` is used in `PARTITION BY` and `transaction_date` in `ORDER BY`, so indexing both together lets the planner access rows in the right order without scanning everything.

C. Result

After the index was added, `EXPLAIN ANALYZE` showed an Index Scan instead. Execution time dropped to about 142 ms, roughly 6x faster. The full before-and-after output is in `dbt_project/performance_report.md`.

TABLE I
QUERY PERFORMANCE BEFORE AND AFTER INDEXING

Metric	Before	After
Scan Type	Sequential Scan	Index Scan
Execution Time	~851 ms	~142 ms
Speedup	–	~6x

VIII. CI/CD PIPELINE

The GitHub Actions workflow lives at `.github/workflows/ci.yml`. It fires on every pull request to main and runs these steps in order:

- 1) Check out the repo.
- 2) Set up Python 3.11.
- 3) Install `dbt-postgres`, `sqlfluff`, and `sqlfluff-templater-dbt`.

- 4) Create the dbt profile at runtime from GitHub Secrets: DB_HOST, DB_USER, DB_PASSWORD, DB_PORT, and DB_NAME.
- 5) Clear SQLFluff cache.
- 6) Lint SQL files in dbt_project/models and dbt_project/analyses with SQLFluff.
- 7) Run dbt deps.
- 8) Run dbt run.
- 9) Run dbt test.

This means any SQL style problems, broken models, or failing tests get caught before anything merges into main. All database connection details are pulled from GitHub Secrets so nothing sensitive is stored in the repo.

IX. GIT WORKFLOW

All of Phase 2 was done on a separate branch (phase2-dbt) with commits broken up by feature, models, tests, queries, the performance report, and the CI/CD config each got their own commits. A pull request was opened and merged into main after review.

X. DELIVERABLES SUMMARY

- dbt_project/models/example/ – dimension models, fact model, and schema tests
- dbt_project/analyses/ – analytical SQL queries
- dbt_project/performance_report.md – EXPLAIN ANALYZE output and indexing results
- star_schema.png – dimensional model diagram
- .github/workflows/ci.yml – GitHub Actions workflow
- .sqlfluff – SQL linting config
- Updated README.md

XI. CONCLUSION

Phase 2 converted the Phase 1 schema into a star schema that's actually usable for analytics, added dbt tests to catch data issues automatically, wrote three analytical queries using CTEs and window functions, and got a key query down from about 851 ms to 142 ms with a single composite index. The CI/CD pipeline now runs linting, model execution, and testing on every pull request so invalid changes do not get merged into main.